encapsulation: why? (1)

some possible scnearios (1/2):

add encryption/authentication to data in flight

be “on University/company network” from home

hide original location of Internet connection

make virtual machines running on different servers appear to be plugged into one switc

encapsulation: why? (2)

some possible scnearios (2/2):

evade overally restrictive firewall rules

make two datacenters connected via Internet appear to be one big network

'separate' networks for phones v. desktops without buying two sets of switches

encapsulation options [incomplete]

left in above	TCP/UDP/higher layers	IP	link-layer
above TCP/UDP	HTTP proxy, DNS over HTTP(S)	—	—
TCP/UDP	SOCKS, HTTP CONNECT, SSH conn forwarding, TLS	—	—
IP	OpenVPN, WireGuard	GRE, IPsec	MPLS
link-layer	OpenVPN, ...	?	VLAN, MPLS

encapsulation steps

1. getting the stuff to encapsulate
2. sending it encapsulated

will start by focusing on first problem

encapsulating w/ app changes

application might have special code to handle connecting differently

\+ might take advantage of extra information in encapsulation

example: many application's TLS support

example: web browser HTTP/SOCKS proxy support

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

Linux tap devices

```
# create virtual ethernet device mydev
$ ip tuntap add dev mydev mode tap
# mark ethernet device as up
$ ip link set mydev up
$ dhclient mydev # or other commands to use/config device
```

(dhclient is a DHCP client)

—

```
int opentap(const char * name) {
    ... /* setup code, not shown*/
}
...
int fd = opentap("mydev");
...
write(fd, ethernetPacket, packetSize)
/* and (probably in separate threads) */
read(fd, buffer, SIZE); processEthernetPacket(buffer);
```

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

Linux tun devices

same as ‘tap’ devices, but...

get IP packets, not ethernet packets

```
# create virtual ethernet device mydev
$ ip tuntap add dev mydev mode tun
# setup device to be routed to, example:
$ ip address add 10.0.0.2 dev mydev
$ ip route add 10.0.0.0/24 dev mydev
$ ip -6 address add 3fff:1234::1 dev mydev
$ ip -6 route add 3fff:1234::/32 dev mydev
```

tunneling program can then open device and read/write IP packets

full tunnel routing table

say 198.51.100.5 is running tunnel sever,
and 10.0.2.5 is gateway beyond tunnel,
and 203.0.113.54 is local gateway:

address	next hop	dev	priority
10.0.2.0/24	—	tunnel	normal
198.51.100.5/32	208.0.113.54	real	high
(default)	203.0.113.54	real	normal
(default)	10.0.2.5	tunnel	high

split tunnel routing table

say 198.51.100.5 is running tunnel sever,
and 10.0.2.5 is gateway beyond tunnel,
and 10.0.0.0/16, 198.51.100.0/24 are tunneled networks
and 203.0.113.54 is local gateway:

address	next hop	dev	priority
10.0.2.0/24	—	tunnel	normal
10.0.0.0/16	10.0.2.5	tunnel	normal
198.51.100.0/24	10.0.2.5	tunnel	normal
198.51.100.5/32	208.0.113.54	real	high
(default)	203.0.113.54	real	normal

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

‘transparent’ proxy (1)

```
ProxyChains README
current version: 3.1
=====================

This is open source software for GNU/Linux systems.

proxychains - a tool that forces any TCP connection made by any given application
to follow through proxy like TOR or any other SOCKS4, SOCKS5 or HTTP(S) proxy.
Supported auth-types: "user/pass" for SOCKS4/5, "basic" for HTTP.
```

configures dynamic library loader to load its library

LD_PRELOAD

with special versions of `connect`

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

socket-in-socket

example: SSH connection forwarding

```
ssh -L X:remotehost:remoteport hostname
```

example: `stunnel` for 'tunneling' TCP in TLS

run on port X, configure to connect to some remote host

configure program to connect to localhost port X

...instead of remote host

encapsulating w/o app changes

generally: easier to do for lower layers

link-layer in something

- "virtual" (probably Ethernet) device
- sends/receives from 'tunnel'

IP in something

- "virtual" IP link
- destination routing table can go to

UDP/TCP in something

- replace socket API
- convince application to connect to different IP address?
- use virtual IP link + implement one end of TCP/UDP

ethernet to socket translation?

know of programs that capture Ethernet, extract socket() operations

examples: slirp, passt

current main use:

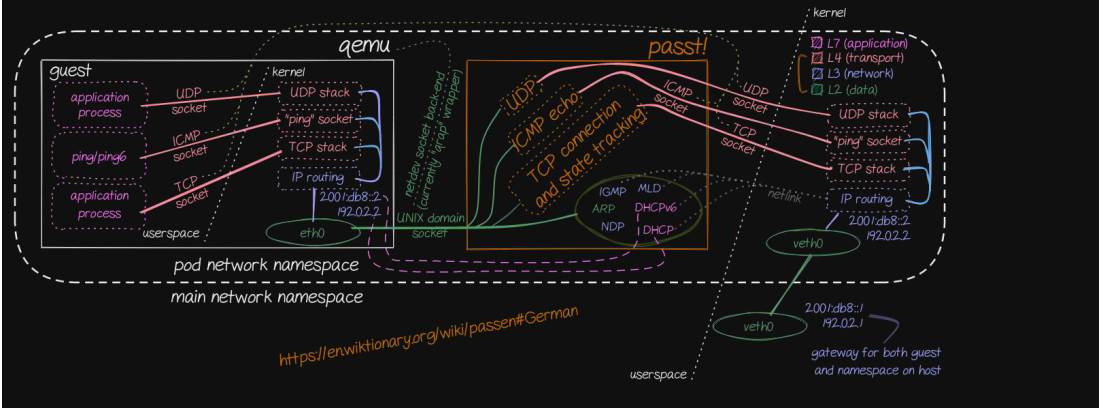
- networking for virtual machines ...
- ...without permission to configure new virtual network devices

but these don't support then tunneling the TCP/UDP connections

- instead just make normal socket calls

passt: Plug A Simple Socket Transport

passt implements a translation layer between a Layer-2 network interface and native Layer-4 sockets (TCP, UDP, ICMP/ICMPv6 echo) on a host. It doesn't require any capabilities or privileges, and it can be used as a simple replacement for Slirp.



backup slides